

Analysis of the Effectiveness of Super-Resolution Algorithms for Improving Computational Results: A Case Study on Two-Dimensional Diffusion Problems with Heterogeneous Coefficients

IVAN V. KONYUKHOV¹ and VLADIMIR M. KONYUKHOV²

¹Research Center of the Artificial Intelligence Institute, Innopolis University, Innopolis, 420500 Russia (e-mail: i.konyukhov@innopolis.ru)

²Research Center of the Artificial Intelligence Institute, Innopolis University, Innopolis, 420500 Russia (e-mail: v.konyukhov.r@innopolis.ru)

Corresponding author: Ivan V. Konyukhov (e-mail: i.konyukhov@innopolis.ru).

The study was supported by the Ministry of Economic Development of the Russian Federation (agreement No. 139-10-2025-034 dd. 19.06.2025, IGK 000000C313925P4D0002).

ABSTRACT In this work, we explored the potential of using super-resolution techniques to reconstruct detailed concentration fields from their representations on coarse grids. We compared the performance of classical interpolation methods, such as bilinear and bicubic, with modern neural network-based approaches. It was demonstrated that intelligent algorithms, in particular those based on residual networks, demonstrated significant superiority in terms of reconstruction quality, as measured by the peak signal-to-noise ratio. These algorithms were also able to accurately reproduce the boundaries of complex geometric structures. Investigating the dependence of training quality on the balance of dataset, the "hallucination" effect of the network was identified and analyzed. We also studied the computational efficiency of these methods, including training time and inference performance, while using central processing units, graphics processing units and neural processing units. It was demonstrated that the multi-scale deep super-resolution method showed the best results according to the criteria of reconstruction quality, training time and inference speed.

INDEX TERMS diffusion, interpolation, machine learning, numerical methods, super-resolution algorithms

I. INTRODUCTION

THE solution to many problems related to modeling complex processes is obtained by using grid methods [1]. The computational complexity of classical grid methods, such as finite difference and finite element methods, increases sharply with the number of spatial variables and required accuracy. This is especially true for two- and three-dimensional unsteady problems, where the demands on computational resources and solution speed become extremely high. Recently developed super-resolution algorithms have made it possible to significantly improve the computer modeling efficiency of complex processes, see for example [2]–[4]. These methods allow us to reconstruct the calculated characteristics fields of the processes studied, which were obtained on coarse computational grids with small number of nodes, into the corresponding high-resolution fields by densifying the grids and increasing their size. By doing so, the computational load is shifted from computationally-extensive numerical modeling

to the inference of a trained neural network and its execution on specialized hardware, such as graphics processors or neural processing units [5].

Note that if the fields are constructed with a high degree of the details on grids with a greater number of nodes, then their representation on grids containing a smaller number of nodes (which obviously leads to a deterioration in their quality) can be achieved by solving a well-posed direct problem

$$LR = \text{deg}(HR), \quad (1)$$

where deg is a degradation function (e.g., due to blurring or noise addition); LR and HR denote low-resolution and high-resolution fields, respectively. The inverse super-resolution problem $HR = \text{deg}^{-1}(LR)$ is the recovering a high-resolution field from its low-resolution version. Generally speaking, this is an ill-posed problem with no exact solution [6].

Classical interpolation algorithms, such as linear and bicubic, are typical examples of super-resolution methods [7],

which preceded the era of neural network algorithms based on deep learning. In particular, in one-dimensional case, linear interpolation allows the reconstruction of function values at intermediate nodes between two adjacent points on a coarse grid using the equation of a straight line. This ensures smoothness of the function up to the first-order derivative, but it can lead to blurring of piecewise continuous boundaries with the derivative discontinuities and loss of the fine details in the fields.

In turn, one-dimensional cubic interpolation uses a dependence constructed from four nodes of the coarse grid. Due to this fact, the reconstructed function is continuous with derivatives up to the second order and retains greater accuracy in intermediate nodes, although it takes more computational resources.

Similar problems arise when using bilinear and bicubic interpolation algorithms in two- and three-dimensional cases. However, despite their simplicity and high speed, these deterministic algorithms have fundamental limitations in their ability to recover high-frequency information that is lost during resolution reduction. This has led to the development of more advanced techniques based on neural networks that can learn complex structural patterns to reconstruct missing details [8].

Currently, a large number of works are devoted to these problems. Without aiming to provide a detailed review and analysis of all of them, we mention some works as examples to demonstrate the variety of the approaches used for solving partial differential equations (PDEs). For instance, a thorough review of the findings on the use of super-resolution techniques can be found in [9].

The first neural network-based super-resolution algorithms used convolutional neural networks (CNNs) [10]. CNNs are a type of deep learning algorithm that is designed to process data with a spatial structure, such as two-dimensional images and three-dimensional scenes. Their key distinction from ordinary fully connected networks lies in the use of convolutional layers, in which a small matrix (kernel) "slides" across the entire input image, computing scalar products and forming feature maps. By doing so, the network automatically extracts hierarchical features, starting from simple edges and textures in the initial layers of the network and progressing to more complex objects in deeper layers. Due to their properties such as translation invariance and efficient use of parameters, CNNs have become the basis for computer vision applications.

The Super-Resolution Convolutional Neural Network (SRCNN), introduced in [10], enhances this approach. The SRCNN operates in three stages: (1) feature extraction, (2) non-linear mapping, and (3) final reconstruction. Unlike classical interpolation, SRCNN learns the convolutional kernel weights from a large dataset to minimize mean squared error (MSE) between the generated and ground truth high-resolution fields.

There are various strategies for using convolutional neural networks in "super-resolution" algorithms, see the review [5]. For example, some methods, such as the Very Deep Super

Resolution (VDSR) algorithm [11], use a preliminary increase in the number of grid nodes (pre-upsampling) to construct a detailed solution. The Fast Super-Resolution Convolutional Neural Network (FSRCNN) algorithm [12] performs a post-increase the grid size (post-upsampling). The Laplacian Pyramid Super-Resolution Network (LapSRN) [13], in turn, uses a consequential increase in grid size (progressive upsampling) in the layers of the neural network, which is especially effective at higher values of the scaling factor.

A fundamental limitation in training deep convolutional neural networks is the "vanishing gradient problem", which is an exponential decrease in gradient magnitude as it propagates through the network, from output layers to input layers through successive layers. This is caused by the fact that gradients are calculated using the chain rule of differentiation, with multiple products of the derivatives, in error backpropagation process. This effect is most commonly seen with activation functions such as the sigmoid and hyperbolic tangent functions, which can lead to weights failing to update in earlier layers of the network, making deep learning inefficient or impossible, and limiting network's practical depth and ability to detect complex features [14].

Residual Networks (ResNets) [15] were specifically designed to solve the vanishing gradient problem in training very deep networks. Modern super-resolution algorithms, including Enhanced Deep Super-Resolution (EDSR) [16] and Multi-scale Deep Super-Resolution (MDSR) [17], are built upon ResNet blocks. These models incorporate skip connections that allow the network to learn the residual, i.e. the difference, between a low-quality interpolated image and the target high-resolution image. The residual blocks primarily encode high-frequency components such as textures, edges and fine details thereby accelerating convergence and significantly improving reconstruction quality especially at high upscaling factors, e.g. $\times 4$, $\times 8$. Modified ResNet variants also show enhanced robustness to various artifacts and noise types, making them highly effective for solving computer vision problems [14].

Such methods can also be effective in solving partial differential equations when the fields of physical characteristics are computed using numerical methods on coarse grids and then reconstructed using super-resolution algorithms on grids with a large number of nodes. This approach may significantly reduce the time required to solve the problem. However, it's necessary to note that if in standard image processing, perceptual quality is the primary metric, and minor structural hallucinations in natural images may be visually acceptable, then the application of super-resolution algorithms for solving differential equations can lead to the significant distortion of the real geometry and non-uniform structure of the solution region that invalidates the numerical results.

Note that new methods for solving PDEs have been developed recently by integrating physical constraints directly into super-resolution architectures. For example, [18] proposes a physics-constrained framework for continuous space-time super-resolution in the Rayleigh-Bénard convection prob-

lem, ensuring consistency with the governing flow equations. Similarly, the paper [19] demonstrates the effectiveness of physics-informed CNNs in reconstructing sparse observations of dynamical systems with chaotic turbulent Kolmogorov flows. It is shown that this method outperforms classical interpolation methods in fine-scale reconstruction of turbulent structures, promising for experimental scenarios with limited measurement resolution.

While physics-constrained methods provide physical consistency and data efficiency, purely data-driven models have certain advantages. In particular, they can learn implicit patterns directly from observations without requiring explicit physical constraints. They are also simpler to implement, train, and infer faster, because they avoid the expensive computation of PDEs residuals via automatic differentiation or finite difference approximations required by physics constrained methods. Therefore, our study focuses on evaluating the efficiency of established data driven architectures as a post processing tool for numerical PDEs solutions.

The goal of this research is to apply these algorithms to solving two-dimensional diffusion problems in regions with complex geometry and non-uniform diffusion coefficients. This work is an application-focused comparison study evaluating the performance of various neural network architectures, as well as identifying the conditions that lead to the "hallucination" effect during model training on imbalanced datasets, and providing practical guidelines for deploying super-resolution techniques on various computational platforms within scientific computing workflows.

Note, that parabolic equations arise not only in modeling diffusion but also in heat conduction, migration of contaminants, multi-phase filtration in non-uniform oil reservoirs, blood flow along the vessels with a porous thrombus on the walls, and other processes, see, for example [20]–[25].

II. FORMULATION OF THE DIFFUSION PROBLEM AND ITS NUMERICAL SOLUTION

A. PROBLEM STATEMENT

Consider a homogeneous linear differential equation of parabolic type in a two-dimensional rectangular domain $\Omega = [0, T] \times [0, X] \times [0, Y]$ of a complex structure, where the unknown function $u(t, x, y)$ satisfies zero boundary conditions on all sides of the rectangle except for the top side and has a zero initial condition:

$$\frac{\partial u}{\partial t} = \frac{\partial}{\partial x} \left(D(x, y) \frac{\partial u}{\partial x} \right) + \frac{\partial}{\partial y} \left(D(x, y) \frac{\partial u}{\partial y} \right), \quad (2)$$

$$0 < x < X, \quad 0 < y < Y, \quad t > 0;$$

$$u(t, 0, y) = 0; \quad u(t, X, y) = 0; \quad (3)$$

$$u(t, x, 0) = 0; \quad u(t, x, Y) = U_Y; \quad (4)$$

$$u(0, x, y) = 0. \quad (5)$$

For convenience in the subsequent discussion, let's assume that the process under consideration is the spreading of the concentration u of a diffusing substance, and $D(x, y)$ is the diffusion coefficient.

B. NUMERICAL METHOD

To test the neural network solution Ψ generated by super-resolution algorithms, we will use the solution of the problem (2)–(5), obtained by the finite difference method [26], [27]. Let us discretize the continuous domain Ω using a uniform grid of a finite set of nodes (t_k, x_i, y_j) and replace the governing equation (2) by an implicit finite-difference scheme with second-order approximation $O(h_t^2 + h_x^2 + h_y^2)$:

$$\frac{u_{i,j}^{k+1} - u_{i,j}^k}{h_t} = \Lambda_x u_{i,j}^{(0.5)} + \Lambda_y u_{i,j}^{(0.5)}, \quad (6)$$

$$k = \overline{0, N_t - 1}, \quad i = \overline{1, N_x - 1}, \quad j = \overline{1, N_y - 1};$$

$$\Lambda_x u_{i,j}^{(0.5)} = \frac{1}{h_x^2} (D_{i-1/2,j} u_{i-1,j}^{(0.5)} -$$

$$- (D_{i-1/2,j} + D_{i+1/2,j}) u_{i,j}^{(0.5)} + D_{i+1/2,j} u_{i+1,j}^{(0.5)}),$$

$$\Lambda_y u_{i,j}^{(0.5)} = \frac{1}{h_y^2} (D_{i,j-1/2} u_{i,j-1}^{(0.5)} -$$

$$- (D_{i,j-1/2} + D_{i,j+1/2}) u_{i,j}^{(0.5)} + D_{i,j+1/2} u_{i,j+1}^{(0.5)}),$$

where $t_k = kh_t$, $x_i = ih_x$, $y_j = jh_y$, $h_t = T/N_t$, $h_x = X/N_x$, $h_y = Y/N_y$, $k = \overline{0, N_t}$, $i = \overline{0, N_x}$, $j = \overline{0, N_y}$; N_t , N_x and N_y are the numbers of nodes along corresponding coordinate axes t , x and y ; $u_{i,j}^k = u(t_k, x_i, y_j)$; $u_{i,j}^{(0.5)} = (u_{i,j}^{k+1} + u_{i,j}^k)/2$; $D_{i\pm 1/2,j} = D(x_i \pm h_x/2, y_j)$; $D_{i,j\pm 1/2} = D(x_i, y_j \pm h_y/2)$.

Following the longitudinal-transverse sweep method [26], we introduce an intermediate time point $\bar{t} = t_{k+0.5}$ and subdivide the scheme (6) into two systems of algebraic equations:

$$\frac{\bar{u}_{i,j} - u_{i,j}^k}{h_t/2} = \Lambda_x \bar{u}_{i,j} + \Lambda_y u_{i,j}^k, \quad (7)$$

$$\frac{u_{i,j}^{k+1} - \bar{u}_{i,j}}{h_t/2} = \Lambda_y u_{i,j}^{k+1} + \Lambda_x \bar{u}_{i,j}. \quad (8)$$

As shown in [26], the longitudinal-transverse scheme (7) – (8) is equivalent to the original implicit scheme (6) and retains the second order $O(h_t^2 + h_x^2 + h_y^2)$ of approximation.

The solution of systems proceeds in two steps. Firstly, the tri-diagonal system (7) is solved with the one-dimensional sweep method along the x -direction, and as a result, the auxiliary grid function $\bar{u}_{i,j}$ is computed at the intermediate time point $\bar{t}$. Secondly, using the obtained $\bar{u}_{i,j}$, the desired solution $u_{i,j}^{k+1}$ is determined from the system (8) along the y -direction.

To maintain second-order accuracy in the first step, the boundary conditions at time $t_{k+0.5}$ for $i = 0$ and $i = N_x$ must be approximated using the following general relationship

$$\bar{u}_{i,j} = \frac{1}{2} (u_{i,j}^{k+1} + u_{i,j}^k) - \frac{h_t}{4} \Lambda_y (u_{i,j}^{k+1} - u_{i,j}^k).$$

However, in our case the boundary conditions (3) are homogeneous, then the correction term $h_t/4 \cdot \Lambda_y (u_{i,j}^{k+1} - u_{i,j}^k)$ vanishes, yielding simply:

$$\bar{u}_{0,j} = 0, \quad \bar{u}_{N_x,j} = 0, \quad j = \overline{0, N_y}.$$

In the second step, approximation of the boundary conditions on $y = 0$ and $y = Y$ pose no difficulty, as they are prescribed on grid nodes t_{k+1} :

$$u_{i,0}^{k+1} = 0, \quad u_{i,N_y}^{k+1} = U_Y, \quad i = \overline{0, N_x}.$$

Note that since the longitudinal-transverse scheme (7) – (8) is uniformly and unconditionally stable with respect to both the initial data and the right-hand sides [26], its convergence rate is $O(h_t^2 + h_x^2 + h_y^2)$.

C. INTERPOLATION AND SUPER-RESOLUTION ALGORITHMS

Usually, for example, in image processing, interpolation formulas, which only require the function values at the grid nodes, are often used.

Bilinear interpolation.

Let $u_{i,j}$ denote the values of the desired discrete scalar function at grid nodes (x_i, y_j) , $i = \overline{0, N_x}$, $j = \overline{0, N_y}$, and $\omega_{i,j}$ is an arbitrary quadrilateral cell defined by vertices (x_i, y_j) , (x_{i+1}, y_j) , (x_i, y_{j+1}) and (x_{i+1}, y_{j+1}) . The value of $u(x, y)$ at any point (x, y) inside $\omega_{i,j}$ can be calculated using the bilinear interpolation formula [7]:

$$u(x, y) = \frac{(y_{j+1} - y)(u_{i,j}(x_{i+1} - x) + u_{i+1,j}(x - x_i))}{(x_{i+1} - x_i)(y_{j+1} - y_j)} + \frac{(y - y_j)(u_{i,j+1}(x_{i+1} - x) + u_{i+1,j+1}(x - x_i))}{(x_{i+1} - x_i)(y_{j+1} - y_j)}. \quad (9)$$

Formula (9) ensures the continuity of the function, but not its derivatives.

Bicubic interpolation. First, we consider the construction of a one-dimensional cubic interpolation polynomial $u(x)$. In this case, the values of the functions $f_0 = u_{i-1}$, $f_1 = u_i$, $f_2 = u_{i+1}$ and $f_3 = u_{i+2}$ are defined not only at the nodes x_i and x_{i+1} , but also at the neighboring points x_{i-1} and x_{i+2} . The introduction of the normalized coordinate $\varsigma = (x - x_i)/(x_{i+1} - x_i)$ establishes a one-to-one correspondence between the points x_{i-1} , x_i , x_{i+1} , x_{i+2} and $\varsigma_0 = -1$, $\varsigma_1 = 0$, $\varsigma_2 = 1$, $\varsigma_3 = 2$. The cubic interpolating polynomial on the interval $[-1, 2]$ can be written as [7]

$$U(f_0, f_1, f_2, f_3, \varsigma) = f_1 + \frac{1}{6} [(-2f_0 - 3f_1 + 6f_2 - f_3)\varsigma + (3f_0 - 6f_1 + 3f_2)\varsigma^2 + (-f_0 + 3f_1 - 3f_2 + f_3)\varsigma^3]. \quad (10)$$

For any point $x \in [x_i, x_{i+1}]$, the interpolated value of the desired function $u(x)$ is obtained as

$$u(x) = U(f_0, f_1, f_2, f_3, (x - x_i)/(x_{i+1} - x_i)).$$

In the two-dimensional case, a bicubic polynomial is constructed similarly to formula (10), using the values of the grid function u at the four nodes of the cell $\omega_{i,j}$ and at the twelve nodes of the neighboring cells $\omega_{i,j\pm 1}$, $\omega_{i\pm 1,j}$, and $\omega_{i\pm 1,j\pm 1}$. Let normalized coordinates be $\varsigma = (x - x_i)/(x_{i+1} - x_i)$ and $\eta = (y - y_j)/(y_{j+1} - y_j)$. We denote the values of function

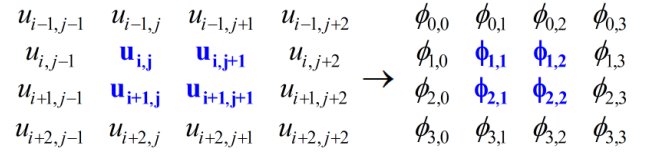


FIGURE 1. Numbering of grid function values in cells $\omega_{i,j}$, $\omega_{i,j\pm 1}$, $\omega_{i\pm 1,j}$, and $\omega_{i\pm 1,j\pm 1}$.

at the nodes of these cells, as shown in Fig. 1. The values corresponding to the central cell $\omega_{i,j}$ are highlighted in bold.

A cubic polynomial of the variable ς is constructed for each row $m = -1, 0, 1, 2$ in a similar way to formula (10):

$$R_m(\varsigma) = U(\phi_{m,0}, \phi_{m,1}, \phi_{m,2}, \phi_{m,3}, \varsigma), \quad m = \overline{0, 3}. \quad (11)$$

Then, interpolation is done in the direction of η using (11):

$$\Phi(\varsigma, \eta) = U(R_0(\varsigma), R_1(\varsigma), R_2(\varsigma), R_3(\varsigma), \eta). \quad (12)$$

Thus, for any point $(x, y) \in \omega_{i,j}$,

$$u(x, y) = \Phi((x - x_i)/(x_{i+1} - x_i), (y - y_j)/(y_{j+1} - y_j)).$$

Note that bicubic interpolation (11)–(12) ensures continuity of the first derivative and provides more accurate results than bilinear interpolation, but requires more computational operations. Although bilinear interpolation is not as accurate as bicubic interpolation, but it is easier to implement and faster to compute.

D. SUPER-RESOLUTION METHODS

We first consider super-resolution algorithms based on Convolutional Neural Networks (CNNs). Their typical architecture is shown in Fig. 2 and consists of the following sequential blocks.

- 1) *Convolutional layers* for extracting features (properties, details, types, etc.) from input data elements.
- 2) *Subsampling (pooling) layers* for reducing the size of processed information while increasing the invariance of extracted features with respect to the location of objects in the input data.
- 3) *Fully connected layers* performing the final function of classification or regression.

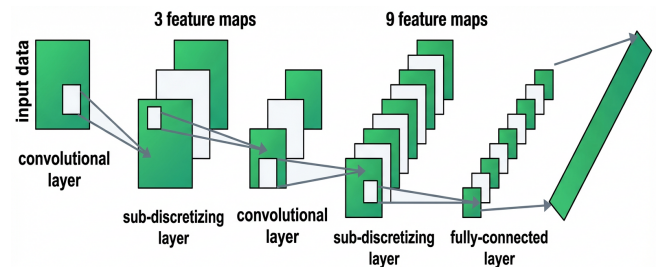


FIGURE 2. Architecture of a convolutional neural network.

One of the most well-known CNN-based super-resolution algorithms is SRCNN (Super-Resolution Convolutional Neural Network), which is described in detail, for example, in [10]. In the preliminary stage of the SRCNN method, the low-resolution (LR) image is first enlarged to the desired size using a specified interpolation algorithm. The resulting image is further denoted as IHR (Interpolated High-Resolution). The goal of the network is to transform the IHR into an SHR (Super-High-Resolution) image that closely approximates the original HR image. This process is realized in three stages.

- 1) **Feature extraction** At this stage, the first layer of the neural network takes an interpolated image as input. This image is then divided into small patches, and convolutional filters are applied to generate high-level feature vectors for each selected area. Rather than using predefined features (for example, gradients), the neural network learns on its own which features are the most useful for subsequent reconstruction. For this layer, you can set the size of the convolution core, which is usually 9×9 ; the number of filters, for instance 64; and the activation function. Most commonly, the Rectified Linear Unit (ReLU) is used in neural networks to introduce non-linearity into the output of each neuron.
- 2) **Non-linear mapping.** The second layer is the most important layer in the neural network. It takes a feature vector from the first layer and transforms it through a non-linear mapping into a feature vector representing a small patch of the IHR image. This layer actually performs the main task of restoring high-frequency details that were missing from in the original low-resolution image. Similar to the previous layer, the size of the convolution core is usually set to 5×5 , and the number of filters is 32. ReLU is also used as the activation function.
- 3) **Final reconstruction.** The third layer of the neural network takes an input vector of IHR image features from the second layer and processes them to create a final SHR image. This layer can be seen as the one that "puzzles together" the restored parts, ensuring the continuity and integrity of the final image. No activation function is typically used in this layer.

The advantages of the SRCNN method include, firstly, its simplicity and fast training of a neural network. Secondly, it significantly surpasses classical interpolation methods in terms of the quality of the resulting images. But also the method has some limitations.

- 1) The neural network does not directly operate with the LR image, but requires its preliminary transformation into an IHR image. This process is computationally inefficient and may introduce noise into the image.
- 2) One network is trained for a specific scaling factor ($\times 2$, $\times 3$, $\times 4$), i.e., each scale requires its own model.
- 3) Due to the small number of layers and their sizes, the network can only analyze small sections of the image at a time, which limits its ability to reconstruct the entire

image.

Note that the SRCNN method formed the basis for all subsequent deep learning-based super-resolution models, such as FSRCNN, VDSR and LapSRN. These models eliminated its limitations and improved image scaling results. Detailed description of the FSRCNN and VDSR methods can be found in [11], [12]. For comparison, Table 1 summarizes the key characteristics of SRCNN and VDSR only, as VDSR actually generalizes capabilities found in FSRCNN and LapSRN.

TABLE 1. Key characteristics of VDSR and SRCNN algorithms

Parameter	VDSR	SRCNN
Network depth	20 convolutional layers	3 convolutional layers
Receptive field	Large (41×41), it takes into account the global context	Small (13×13), it is limited to the local context
Training speed	High, due to residual learning and gradient clipping	Slower than VDSR
Quality	Very high, especially at large scales	Good baseline, but inferior to VDSR
Scale support	Single model for multiple scales ($\times 2$, $\times 3$, $\times 4$)	Separate model per scale

A common limitation of the above methods is the *vanishing gradient problem*, which can hinder the deep training of neural networks. As the network depth increases (e.g., to 50, 100 layers or more), the gradients that are transmitted back during training can become extremely small. This effectively stops the learning process. Modern approaches like EDSR and MDSR are based on residual ResNet blocks [16], [17] and are devoid of this disadvantage. The ResNet architecture allows you to create models in hundreds and thousands of layers without performance degradation.

The main idea behind ResNet is to teach the *residual function* $F(\chi)$ using residual blocks instead of a direct mapping $H(\chi)$, that is

$$\mathcal{F}(\chi) = H(\chi) - \chi,$$

where χ is the block input. This is achieved by using skip connections, which bypass one or more layers. The block output is then $H(\chi) = \mathcal{F}(\chi) + \chi$. Such architecture of ResNet allows gradients to flow directly through the network during the error backpropagation, and makes it possible to effectively train networks with a depth of hundreds or even thousands of layers.

A typical residual block contains 3×3 convolutional layers. In this block, the batch normalization and the ReLU activation function are implemented. Bypass connections are organized between such blocks. If the input and output sizes are different, the 1×1 convolutional layer is used in the skip connection to equalize these values.

A similar neural network architecture takes place in the implementation of the EDSR and MDSR methods. In these models, the skip connections enable the network to focus on identifying differences (i.e., textures, edges, fine details) between the interpolated LR image and the target HR image.

This accelerates convergence and significantly improves reconstruction quality, especially at high upscaling factors

($\times 4$, $\times 8$). Comparative characteristics of the methods are given in Table 2.

TABLE 2. Key characteristics of EDSR and MDSR algorithms

Parameter	EDSR	MDSR
Primary purpose	Single-scale SR (separate model per scale)	Multi-scale SR (single model for multiple scales)
Architecture	Deep residual network (32 blocks, 256 channels); batch normalization removed	Extended EDSR with scale-specific modules (two 5×5 residual blocks at input/output)
Number of parameters	Separate model of size N for each scale $\times 2$, $\times 3$, $\times 4$	Single model of size $\approx N$ for all scales ($3\times$ more parameter-efficient)
Memory efficiency	High (no batch norm)	Very high (shared parameters across scales)
Quality	State-of-the-art for single-scale	Comparable to individual EDSR models per scale
Inference speed	Fast	Slower due to scale-specific modules

In the context of partial differential equations (PDEs), HR and SHR data represent physical fields that are calculated via numerical methods and super-resolution algorithms, respectively. In the considered diffusion problem (2)–(5), these data are the concentration fields $u(t, x, y)$ of the diffusing substance. To solve this problem and compare the results of our computational experiments, we use the currently most effective VDSR and MDSR methods. Hereinafter, VDSR and MDSR are compared with the basic super-resolution algorithm (SRCNN), bilinear and bicubic interpolation methods.

E. METRICS FOR EVALUATING THE QUALITY OF RECONSTRUCTED FIELDS

The quality of the reconstructed HR and SHR fields is evaluated with two standard metrics.

1) **Mean Squared Error (MSE)** is

$$MSE = \frac{1}{N_x N_y} \sum_i^{N_x} \sum_j^{N_y} (u_{i,j}^{HR} - u_{i,j}^{SHR})^2. \quad (13)$$

2) **The Peak Signal-to-Noise Ratio (PSNR)** is a widely applied metric for objectively assessing of the quality of reconstructed images, particularly in the tasks of image compression and super-resolution:

$$PSNR = 10 \log_{10} \left(\max_{i,j}^2 (u_{i,j}^{HR}) / MSE \right). \quad (14)$$

The correspondence between PSNR values and the quality of the reconstructed SHR image is summarized in Table 3, based on the empirical findings presented in [28].

TABLE 3. Interpretation of PSNR values

PSNR value	Image quality	Visual characteristics
> 40 dB	Excellent	Visually indistinguishable from the original; noise is virtually absent
30–40 dB	Good	High quality with minor distortions
20–30 dB	Fair	Noticeable but acceptable distortions
< 20 dB	Poor	Strong distortion and poor quality

It is important to note that, while PSNR and MSE provide standard measures of structural and numerical fidelity between reconstructed and reference fields, they do not inherently enforce physical conservation laws or guarantee satisfaction of the underlying PDE residuals. Consequently, they should be interpreted primarily as indicators of spatial reconstruction accuracy rather than strict physical correctness.

F. NEURAL NETWORK TRAINING METHODOLOGY

To prepare datasets for training various super-resolution neural models, we generate paired LR and HR samples by solving the diffusion problem on the grids with small and large numbers of nodes under diverse initial data. Let us illustrate this process on the example of preparing spatial maps of the diffusion coefficient $D(x, y)$, which are used to compute concentration fields $u(t, x, y)$.

Since the accuracy of difference schemes on the LR-grids is lower than on HR-grids, it is advisable to use not just a subdiscretization approach for model training when the concentration values at the nodes of the LR-grid are taken from the HR-solution. Instead, it should be done by comparing the LR and HR numerical solutions. These paired concentration maps, corresponding to the same time instants t but generated for different spatial distributions of $D(x, y)$ (which play the role of textures), constitute the training dataset.

To ensure a fair comparison, all algorithms are trained on the same fixed dataset consisting of samples with varying geometric structures (or textures).

We consider the following variants of the model distribution $D(x, y)$ within the heterogeneity region ω consisting of both straight and curved boundaries in the diffusion problem (2)–(5).

- 1) $D(x, y) = D_0 = \text{const}$ for $(x, y) \in \Omega$. This variant allows the neural network model to be trained for creating a solution in the homogeneous diffusion domain Ω without any heterogeneous inclusions.
- 2) The diffusion coefficient is piecewise constant:

$$D(x, y) = \begin{cases} D_0, & (x, y) \in \Omega \setminus \omega, \\ D_1, & (x, y) \in \omega, \end{cases}$$

where $D_1 \neq D_0$; $\omega \subset \Omega$ is a geometric inclusion of varying shape (e.g., circle, triangle, square). These configurations are designed to train the network for accurate reconstructing sharp boundaries of heterogeneous domains of different geometries (see Fig. 3(a)).

- 3) The region ω has a different geometric shape and a spatial layered heterogeneous structure with respect to the diffusion coefficient:

$$D(x, y) = \begin{cases} D_0, & (x, y) \in \Omega \setminus \omega, \\ D_1(x, y), & (x, y) \in \omega, \end{cases}$$

where the level lines of the function $D_1(x, y)$ repeat the shape of $\partial\omega$. The function $D_1(x, y)$ may either increase (Fig. 3(b)) or decrease (Fig. 3(c)) from the center of

region ω towards its boundary. In the figures, darker-to-lighter color transitions correspond to increasing values of $D_1(x, y)$.

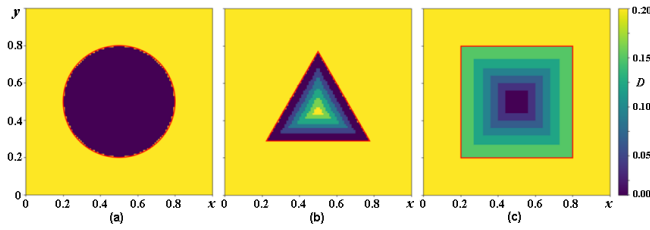


FIGURE 3. Examples of heterogeneity regions ω with different structures: (a) case 2; (b) and (c) case 3.

Since $D(x, y)$ is a table function defined in Ω and $\omega \setminus \Omega$, the resulting concentration field $u(t, x, y)$ inherits the structural features of $D(x, y)$. This is illustrated in Fig. 4.

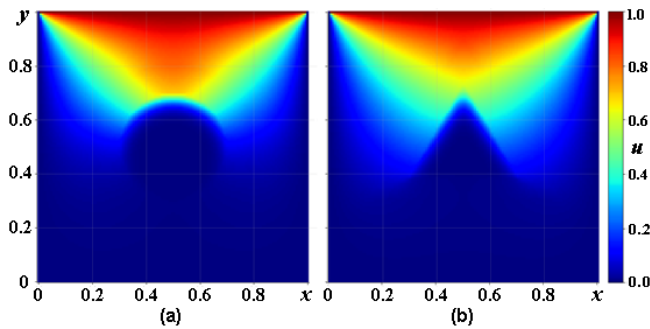


FIGURE 4. Concentration fields $u(x, y)$ corresponding to circular and triangular inclusions ω which are shown in Fig. 3(a) and 3(b), respectively.

Generating LR–HR pairs requires repeatedly solving the diffusion problem, which can be computationally intensive. In addition, finite-difference implementations in interpreted languages such as Python often involve deeply nested loops, significantly limiting performance. To accelerate computations, the *Just-In-Time* (JIT) compiler library *Numba* can be used [29]. It translates the Python modules and NumPy-based mathematical expressions into optimized machine code at runtime, yielding speedups of $\sim 100 - 10000$ times.

Due to the JIT compilation and parallelization features, *Numba* significantly reduces the dataset preparation time from hours to minutes, allowing for training on large and various datasets. For instance, solving the diffusion problem on a grid of $16 \times 16 \times 256$ nodes takes only 0.007 seconds using *Numba*, as opposed to 10 seconds without it, resulting in a speedup of more than 1300 times.

As shown in [8], the loss function $L(\vec{\theta})$ for training of the super-resolution models on dataset $\vec{\theta}$ is usually calculated as the mean squared error between HR- and SHR-images, which in our case are concentration fields $u_{i,j}^{HR}$ and $u_{i,j}^{SHR}$, respectively. Then $L(\vec{\theta}) = \text{MSE}(\vec{\theta})$, where MSE is defined by the formula (13). The optimization problem

$$\min_{\vec{\theta}} L(\vec{\theta}) \quad (15)$$

is solved using the *Adaptive Moment Estimation* (Adam) optimizer [30], which outperforms other methods both in terms of convergence speed and computational efficiency.

Note that minimizing the loss (15) is mathematically equivalent to maximizing the PSNR, which is the primary quality metric for the obtained images. Solving the diffusion problem, the reconstructed concentration fields $u(t, x, y)$ play the role of these images.

III. RESULTS OF COMPUTATIONAL EXPERIMENTS

All computational experiments were carried out in Jupyter notebooks using the Python programming language and the PyTorch deep learning framework [31]. Bilinear and bicubic interpolation was performed using the `scipy.interpolate.interp2d` function from the SciPy library [32].

The computer was equipped with an Intel Core Ultra 125H processor, featuring an integrated Intel Arc Graphics processor unit (GPU) of Alchemist generation and an Intel AI Boost neural processing unit (NPU). To support this hardware, the Intel PyTorch Extension [33] and the Intel NPU Acceleration Library [34] were used within the PyTorch ML-framework.

For the comparison of super-resolution algorithms of the different architectures, the hyperparameters (kernel sizes, numbers and sizes of residual blocks, numbers and dimensions of convolutional layers, etc.) were chosen in a way that the total number of the trainable parameters was approximately the same for all the models ($\sim 5 \cdot 10^5$).

A. PREPARATION OF TRAINING DATASETS

As is known [35], if the training datasets for convolutional neural networks are unbalanced, the hallucination effect can occur. When a model is heavily skewed toward certain classes or visual patterns, it over-optimizes its filters for dominant features while neglecting rare or subtle signals. As a result, processing images with underrepresented objects or atypical structures, the network can "confidently" perceive and classify non-existent elements, producing false positives that resemble cognitive hallucinations, as it projects learned statistical templates onto regions where they are objectively absent.

It's interesting to investigate this effect using the example of a numerical solution to the diffusion equation with heterogeneous coefficients.

To ensure high-quality learning, the training dataset should contain a sufficient number of field pairs with low- and high-resolution, so that the neural network can reliably determine the different geometrical shapes of the non-uniform scope ω of the function $D_1(x, y)$ definition. Specifically, if the number of training examples for circular inclusions is too small compared to other shapes, such as triangles or squares, the network may "hallucinate" and incorrectly reconstruct a circular domain as a triangle or square, simply because it has seen more examples of these shapes during training.

The hallucination effect is illustrated in Fig. 5. In this example, the training set contains $N_{\circ} = N_{\square} = 256$ pairs of circular and square inclusions and only $N_{\triangle} = 16$ pairs

of triangular inclusions. This number N_Δ is not sufficient for accurate shape recognition.

Computational experiments confirmed that, when the numbers of training pairs, N_o , N_Δ , and $N_\square$, are approximately balanced, the network correctly identifies the shape of the non-uniform domain ω in all cases.

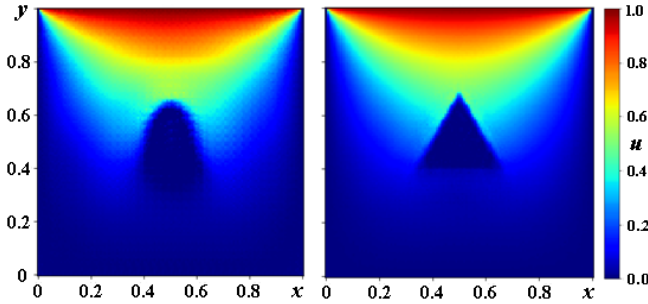


FIGURE 5. Example of the neural network “hallucination” in reconstructing the concentration field $u(t, x, y)$ at solving the diffusion problem with the triangular inclusion ω : a – $N_\Delta = 16$; b – $N_\Delta = 256$.

Figure 6 shows the time evolution of the PSNR metric. Curve (1) corresponds to the imbalanced case ($N_o = N_\square = 256$, $N_\Delta = 16$), where the super-resolution algorithm cannot recognize the triangular shape. The curve (2) shows a balanced situation ($N_o = N_\Delta = N_\square = 256$) when the network correctly reconstructs all three shapes of the non-uniform domain ω .

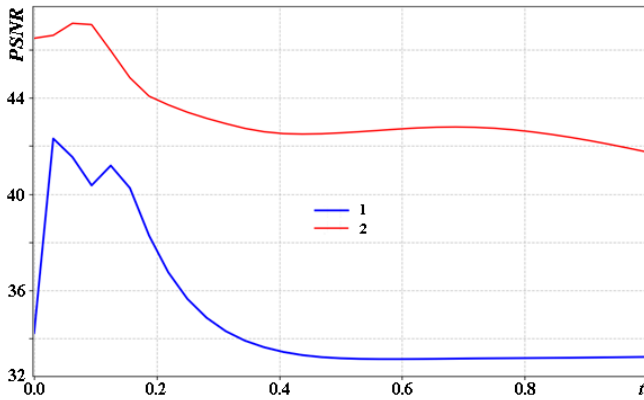


FIGURE 6. PSNR vs. time t : (1) $N_o = N_\square = 256$, $N_\Delta = 16$; (2) $N_o = N_\Delta = N_\square = 256$.

The results presented below are obtained from following four different types of training datasets.

- 1) The heterogeneity region ω is absent. The diffusion coefficient is constant: $D(x, y) = D_0 = \text{const}$ for $(x, y) \in \Omega$.
- 2) A heterogeneity region $\omega \subset \Omega$ with constant diffusion coefficient $D_1 \neq D_0$ is present. The shape of ω can take on various geometric forms, such as a circle, a triangle, or a square.
- 3) The region ω has a spatially layered heterogeneous structure with $D_1 = D_1(x, y)$. The shape of the level

lines of $D_1(x, y)$ is the same as the boundary of ω , and the values of D_1 decrease from the center of ω towards its outer boundary.

- 4) The structure of ω is similar to that in case (III), but the values of $D_1(x, y)$ increase from its center towards the outer boundary.

In all LR- and HR field pairs, the upscaling factor is $\times 4$.

B. TRAINING SPEED OF NEURAL NETWORKS

Let T_1 , T_C and T_X denote the training times of a neural network using a CPU in single-threaded mode, in multi-threaded mode with 18 threads, and using a graphics accelerator (XPU), respectively. To quantify the efficiency of different hardware configurations, we define speedup factors $\kappa_C = T_C/T_1$ and $\kappa_X = T_X/T_1$. Table 4 presents the values $\kappa_C = \kappa_C(N^*)$ and $\kappa_X = \kappa_X(N^*)$ for the aforementioned processor types operating in multi-threaded mode, for various training dataset sizes N^* (in megabytes) and for a fixed number of training epochs $N_e = 20$.

TABLE 4. The efficiency of neural network training for different conditions of solving the optimization problem (15)

Method	N^* , MB	κ_C	κ_X
SRCNN	2	3.6	9.2
SRCNN	4	3.8	11.1
SRCNN	8	4.0	10.3
VDSR	2	2.9	9.4
VDSR	4	3.0	9.8
VDSR	8	3.0	10.4
MDSR	2	2.9	8.1
MDSR	4	2.9	9.5
MDSR	8	3.2	10.8

It is easy to see that, as the training dataset size N^* increases, using an XPU provides approximately 11-fold and 3-fold speedups compared to single-threaded and multi-threaded CPU execution, respectively.

C. CONVERGENCE OF SUPER-RESOLUTION ALGORITHMS

To evaluate the training efficiency of the selected SRCNN, VDSR, and MDSR methods, computational experiments were carried out with network architectures identical to those used in the previous section. In all cases, training was continued until the loss function $\mathcal{L}$ (15) reached a target value of 10^{-4} . The results obtained using XPU, which demonstrated the highest efficiency compared to the single-threaded and multi-threaded CPU modes, are summarized in Table 5, where T_t is the total training time.

TABLE 5. Convergence of super-resolution algorithms

Method	N_e	T_t , s
SRCNN	156	18
VDSR	204	56
MDSR	115	32

The results show that MDSR achieves the target accuracy in solving of the minimization problem with the smallest number of epochs, $N_e = 115$. The training time T_t for

SRCNN and MDSR is less than that of VDSR, which is the least efficient method.

Table 6 illustrates the inference time (in milliseconds) required to generate a single frame, i.e. the concentration field of size 128×128 , from an input 32×32 low-resolution field, using various super-resolution algorithms and hardware platforms.

TABLE 6. Inference time (ms) for generating a single frame of size 128×128

Method	CPU	XPU	NPU
SRCNN	~6	~4	~7
VDSR	~17	~10	~20
MDSR	~12	~7	~15

Although MDSR has a more complicated architecture than VDSR and achieves better performance on fixed-size grids, it is slower than the simpler SRCNN model.

As expected, the XPU is the most efficient inference platform, and the inference times for the single frame generation on the CPU and NPU are quite comparable.

It is worth noting that generating a single frame of the same size using the bicubic interpolation algorithm takes about ~ 3 ms, as it does not require multiple data transformations, like those found in neural networks with more complex structures.

D. EFFECTIVENESS OF SUPER-RESOLUTION ALGORITHMS

Similar conclusions take place for methods' assessment using the PSNR metric, regardless of the domain geometric shape Ω or the structure non-uniformity within the ω .

Figure 7 shows, as an example, the time evolution of $\text{PSNR}(t)$ for case (III) with a triangular shape for the non-uniform region ω and a scaling factor of $\times 4$. All the models were trained on the same dataset, totalling approximately 26 MB.

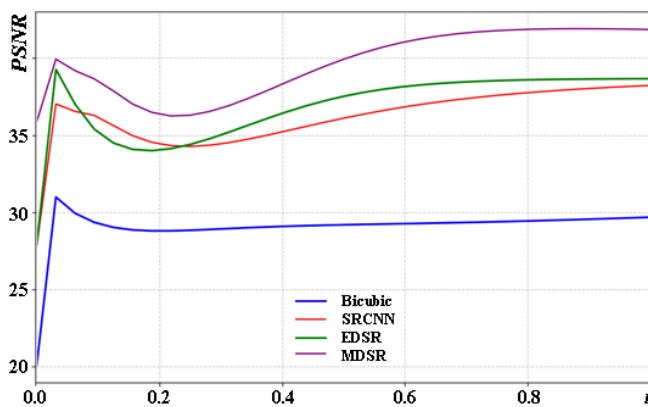


FIGURE 7. PSNR vs. time t for bicubic interpolation and SRCNN, VDSR and MDSR methods.

Notably, as shown by the analysis of the results of multi-variant computational experiments, all neural network algorithms are "good" or "excellent" according to the classification of Table 3 at reproducing the complex outer boundary of inclusion ω under its arbitrary location in the domain Ω ,

unlike the bicubic interpolation algorithm that is estimated as just "fair". The bilinear interpolation algorithm shows "poor" results, its is located significantly lower ($\text{PSNR} < 25$) than the bicubic one and is not presented in this figure.

Figure 8 compares the error fields $\delta_{i,j} = |u_{i,j}^{\text{HR}} - u_{i,j}^{\text{SHR}}|$, where $u_{i,j}^{\text{HR}}$ is the high-resolution numerical solution (128×128) and $u_{i,j}^{\text{SHR}}$ is the reconstructed field obtained by the bicubic interpolation or the MDSR method. Here $i = \overline{1, N_x - 1}$; $j = \overline{1, N_y - 1}$. For visual comparison, the corresponding reconstructed concentration fields are shown in Fig. III-D.

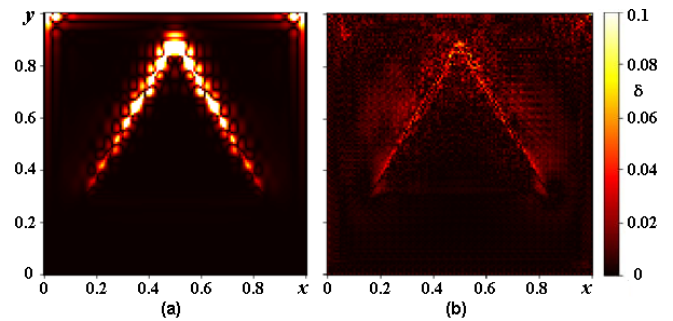


FIGURE 8. Error fields over the diffusion region at final time: a – bicubic interpolation; b – MDSR method.

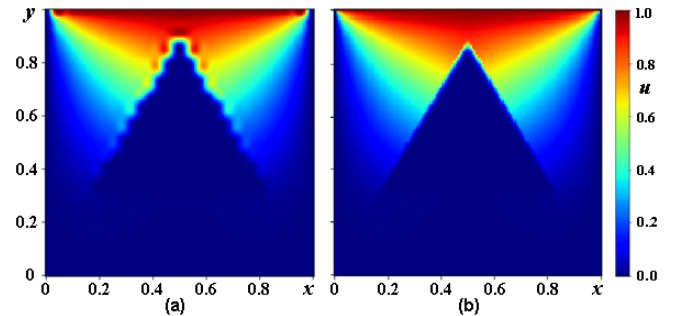


FIGURE 9. Reconstructed concentration fields over the diffusion region at final time: a – bicubic interpolation; b – MDSR method.

As can be seen in Figs. 8 and III-D, the neural network-based algorithm produces significantly more accurate results across the entire domain and in particular, it faithfully reproduces the external boundaries of the heterogeneous region ω .

An important observation from the extensive computational experiments is that, in contrast to bicubic interpolation, all neural super-resolution algorithms robustly and accurately reconstruct the boundaries of complex-shaped inclusions ω , regardless of their location within the domain Ω .

IV. CONCLUSIONS AND FUTURE WORK

Based on the analysis of the results of multi-variant computational experiments, the following conclusions can be drawn.

- 1) Preparing training datasets, it is important to maintain a balanced proportion between the number low-

and high-resolution pairs of concentration fields corresponding to different geometric shapes of heterogeneity regions within the computational area. This balance helps protecting the neural network from the "hallucination" effect, which is its tendency to reduce a problem to the situation where many more pairs of low- and high-resolution images have been used for training.

- 2) The MDSR method requires the fewest training epochs to achieve target minimization accuracy. Both SRCNN and MDSR have shorter training times compared to VDSR, which is the least efficient approach among the considered methods.
- 3) As the training dataset size increases, the use of XPU results in a speedup of approximately 11 and 3 times compared to a single-threaded and multi-threaded CPU, respectively.
- 4) Due to its more advanced architecture, MDSR demonstrates better performance in generating a field of concentration values on a fixed-size grid compared to VDSR, although it remains slower than the simpler SRCNN model.
- 5) The XPU graphics accelerator is the most efficient in terms of frame generation time, while CPU and NPU frame generation times are approximately the same. It should be noted that the use of the bicubic interpolation algorithm takes approximately 3 ms to generate a single frame at the same resolution, as it does not require the multiple data transformations that are typical for complex neural networks.
- 6) All neural network-based super-resolution algorithms achieve "good" to "excellent" quality in reconstructing the boundaries of complex-shaped heterogeneous regions, regardless of their location within the solution area. In contrast, the bicubic interpolation algorithm provides only "satisfactory" results. While such image metrics characterize the quality of field reconstruction, in future studies we will incorporate physics-informed loss functions, namely, direct evaluation of PDE residuals and conservation properties, to ensure physical consistency.

Our future research will primarily focus on analyzing the filtration flow of blood in a blood vessel when a thrombus is present on its wall. The shape and size of the thrombus can dynamically change as a result of its destruction, and this process can be described within the framework of a single-phase filtration model by a differential equation of parabolic type with a non-linear absolute permeability coefficient. In this case, the material of the blood clot is considered as an inhomogeneous porous medium, whose permeability depends not only on space and time coordinates, but also on pressure gradients. The outer boundary of the clot depends on spatial and temporal coordinates. The region of the vessel without a thrombus is treated as a medium with constantly high permeability.

In the periodic mode of the heart, the non-stationary nature

of blood flow filtration within a thrombus can lead to the destruction of areas where the pressure differences become greater than a critical value. Those areas lose their ability to resist shear forces and detach from the thrombus. These moving fragments can lead to a blockage of vessels with smaller diameters, cutting off the blood flow to the heart or brain. Studying such dangerous situations will help us gain a better understanding of their causes and potential consequences. Using super-resolution algorithms can significantly speed up computation of the filtration process providing more detailed information about blood flow in the vessel and areas of blood clots formation.

In addition, super-resolution algorithms can also be applied to solving the problems of contaminant migration in aquifers and multi-phase filtration in complex oil reservoirs. These methods will improve the accuracy of mathematical models and increase the effectiveness of implementing the corresponding algorithms.

ACKNOWLEDGMENT

The study was supported by the Ministry of Economic Development of the Russian Federation (agreement No. 139-10-2025-034 dd. 19.06.2025, IGK 000000C313925P4D0002).

V. ABBREVIATIONS

The following abbreviations are used in this manuscript:

Adam	Adaptive Moment Estimation
CNN	Convolutional Neural Networks
CPU	Central Processing Unit
FSRCNN	Fast Super-Resolution Convolutional Neural Network
GPU	Graphics Processing Unit
HR	High-Resolution
IHR	Interpolated High-Resolution
LapSRN	Laplacian Pyramid Super-Resolution Network
LR	Low-Resolution
MaxSE	Maximal Square Error
MDSR	Multi-Scale Deep Super-Resolution
MSE	Mean Squared Error
NPU	Neural Processing Unit
PDE	Partial Differential Equation
PSNR	Peak Signal-to-Noise Ratio
ReLU	Rectified Linear Unit
ResNet	Residual Networks
SHR	Super-High-Resolution
SRCNN	Super-Resolution Convolutional Neural Network
VDSR	Very Deep Super-Resolution

REFERENCES

- [1] A. Tikhonov, A. Samarskii. *Equations of Mathematical Physics*; Moscow University Press: Moscow, Russia, 2004; 798 p.
- [2] J. Pathak, M. Mustafa, K. Kashinath, E. Motheau, T. Kurth, M. Day. "Using machine learning to augment coarse-grid computational fluid dynamics simulations," arXiv, Oct. 2020, pp. 1-8, DOI: 10.48550/arXiv.2010.00072.
- [3] K. Fukami, K. Fukagata, K. Taira. "Machine-learning-based spatio-temporal super resolution reconstruction of turbulent flows," *J. Fluid Mech.*, vol. 909, A9, Feb. 2021. DOI: 10.1017/jfm.2020.948.
- [4] K. Bao, X. Zhang, W. Peng, W. Yao. "Deep learning method for super-resolution reconstruction of the spatio-temporal flow field," *Adv. Aerodyn.* vol. 5, no. 1, Oct. 2023. DOI: 10.1186/s42774-023-00148-y.
- [5] Z. Wang, J. Chen, S.C.H. Hoi. "Deep Learning for Image Super-Resolution: A Survey," *IEEE Trans. Pattern. Anal. Mach. Intell.*, vol. 43, pp. 3365-3387, Oct. 2021. DOI: 10.1109/TPAMI.2020.2982166.

- [6] W. Yang, X. Zhang, Y. Tian, W. Wang, J.-H. Xue, Q. Liao. "Deep Learning for Single Image Super-Resolution: A Brief Review," *IEEE Trans. Multimed.*, vol. 21, iss. 12, pp. 3106–3121, 2019. DOI: 10.1109/TMM.2019.2919431.
- [7] P. Getreuer. "Linear methods for image interpolation," *Image Processing On Line*, vol. 1, pp. 238–259, Sept. 2011. DOI: 10.5201/ipol.2011.g_lmii.
- [8] C. Weiqin, C. Bo, D. Yuandan, T. Yi, M. Yanling, Z. Juntao. "A review of deep learning-based super-resolution," in 2023 20th International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP), Chengdu, China, Dec. 2023, pp. 1–5. DOI: 10.1109/ICCWAMTIP60502.2023.10387044.
- [9] R. Bin, H. Guo, L. Sun, Z. Wu, T. Radu, Y. Li, et. "The tenth NTIRE 2025 efficient super-resolution challenge report," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition*, Apr. 2025, pp. 1–50. arXiv:2504.10686
- [10] C. Dong, C.C. Loy, K. He, X. Tang. "Image Super-Resolution Using Deep Convolutional Networks," in *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 2, pp. 295–307, Feb., 2016. DOI: 10.1109/TPAMI.2015.2439281.
- [11] J. Kim, J.K. Lee, K.M. Lee. "Accurate Image Super-Resolution Using Very Deep Convolutional Networks," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, June 2016, pp. 1646–1654. DOI: 10.1109/CVPR.2016.182.
- [12] C. Dong, C.C. Loy, X. Tang. "Accelerating the Super-Resolution Convolutional Neural Network," in *Computer Vision – ECCV 2016*, Springer, Cham, Sept. 2016, pp. 391–407. DOI: 10.1007/978-3-319-46475-6_25.
- [13] W.-S. Lai, J.-B. Huang, N. Ahuja, M.-H. Yang. "Fast and Accurate Image Super-Resolution with Deep Laplacian Pyramid Networks," in *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 11, pp. 2599–2613, 2019. DOI: 10.1109/TPAMI.2018.2865304.
- [14] K. He, X. Zhang, S. Ren, J. Sun. "Deep Residual Learning for Image Recognition," in *2016 IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, June 2016, DOI: 10.1109/CVPR.2016.90.
- [15] Y. Zhang, Y. Tian, Y. Kong, B. Zhong, Y. Fu. "Residual dense network for image super-resolution," in *2018 IEEE/CVF Conf. on Computer Vision and Pattern Recognition*, Salt, Lake City, UT, USA, pp. 2472–2481, June 2018. DOI: 10.1109/CVPR.2018.00262.
- [16] B. Lim, S. Son, H. Kim, S. Nah, K. Lee. "Enhanced deep residual networks for single image super-resolution," arXiv, July 2017, pp. 1132–1140. arXiv:1707.02921. DOI: 10.1109/CVPRW.2017.151.
- [17] S. Gao, X. Zhuang. "Multi-Scale Deep Neural Networks for Real Image Super-Resolution," in *Proc. of the 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Long Beach, CA, USA, Apr. 2019, pp. 2006–2013. DOI: 10.1109/CVPRW.2019.00252.
- [18] C. Jiang, S. Esmailzadeh, K. Azizzadenesheli, K. Kashinath, M. Mustafa, H. Tchelepi, P. Marcus, Prabhat, A. Anandkumar. "MeshfreeFlowNet: A Physics-Constrained Deep Continuous Space-Time Super-Resolution Framework," arXiv, May 2020, pp. 1–13. arXiv:2005.01463. DOI: 10.48550/arXiv.2005.01463.
- [19] D. Kelshaw, G. Rigas, L. Magri. "Physics-Informed CNNs for Super-Resolution of Sparse Observations on Dynamical Systems," arXiv, Oct 2022, pp. 1–5. arXiv:2210.17319. DOI: 10.48550/arXiv.2210.17319.
- [20] A.N. Chekalin, M.G. Khranchenkov, V.M. Konyukhov. "Numerical model of rainwater runoff over the catchment surface and transport of contaminant incoming to water stream from soil," *J. of Fundam. and Appl. Sci.*, vol. 9, iss. 25, pp. 890–902, 2017, WOS. 000419757700005.
- [21] V.M. Konyukhov, I.V. Konyukhov, A.N. Chekalin. "Numerical modeling and parallel computations of heat and mass transfer during physical and chemical actions on the non-uniform oil reservoir developing by system of wells," *Computer Research and Modeling*, vol. 12, no. 2, pp. 319–328, 2020. DOI: 10.20537/2076-7633-2020-12-2-319-328.
- [22] V.M. Konyukhov, I.V. Konyukhov, A.R. Ganieva. "Calculation of interrelated thermal processes in a submersible electric motor, rocks and water-gas-oil flow in a producing well," *Program Systems: Theory and Applications*, vol. 13, no. 3(54), pp. 179–194, 2022. DOI: 10.25209/2079-3316-2022-13-3-179-191.
- [23] I.V. Konyukhov, V.M. Konyukhov, M.G. Khranchenkov. "Mathematical modeling of the transient processes in the blood flow along a vessel at the presence of circular clot," *Lobachevskii Journal of Mathematics*, vol. 44, no. 12, pp. 5332–5340, 2023. DOI: 10.1134/S1995080223120235.
- [24] I.V. Konyukhov, V.M. Konyukhov, A.V. Kurdyukov. "Analysis of the Physics-Informed Neural Network Approach to Solving Diffusion Equation," *Lobachevskii Journal of Mathematics*, vol. 46, pp. 1860–1870, 2025. DOI: 10.1134/S1995080225605788
- [25] G.I. Barenblatt, V.M. Entov, V.M. Ryzhik. "Theory of fluid flows through natural rocks," Springer Dordrecht, 1990; 400 p.
- [26] A.A. Samarskii. "An Introduction to the Theory of Difference Schemes," Moscow: Nauka, 197; 552 p.
- [27] E. Süli, D.F. Mayers. "An introduction to numerical analysis," Cambridge: Cambridge University Press, 2003; 433 p.
- [28] Z. Wang, A.C. Bovik, H.R. Sheikh, E.P. Simoncelli. "Image quality assessment: from error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 1–14, Apr. 2004. DOI: 10.1109/TIP.2003.819861.
- [29] S.K. Lam, A. Pitrou, S. Seibert. "Numba: A LLVM-based Python JIT Compiler," in *Proc. of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC '15)*, ACM: New York, Nov. 2015, pp. 1–6. DOI: 10.1145/2833157.2833162.
- [30] D.P. Kingma, J. Ba. "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. for Learning Representations (ICLR-2015)*, San Diego, 2015, pp. 1–15. arXiv:1412.6980.
- [31] A. Paszke, S. Gross, F. Massa. "Pytorch: An imperative style, high-performance deep learning library," in *33rd Conference on Neural Information Processing Systems (NeurIPS 2019)*, Vancouver, Canada, 2019, pp. 1–12. DOI: 10.48550/arXiv.1912.01703.
- [32] R. Johansson. "Interpolation: Numerical Python: Scientific Computing and Data Science Applications with NumPy, SciPy and Matplotlib," Apress: Berkeley, CA, 2019; 723 p. DOI: 10.1007/978-1-4842-4246-9.
- [33] Introducing the Intel Extension for PyTorch for GPUs. (2024). Available online: <https://www.intel.com/content/www/us/en/developer/articles/technical/introducing-intel-extension-for-pytorch-for-gpus.html> (accessed on 25 December 2025).
- [34] Intel NPU Acceleration Library documentation. (2024). Available online: <https://intel.github.io/intel-npu-acceleration-library/index.html> (accessed on 25 December 2025).
- [35] M. Buda, A. Maki, M.A. Mazurowski. "A systematic study of the class imbalance problem in convolutional neural networks," arXiv, Oct 2017, pp. 1–21. arXiv:1710.05381. DOI: 10.48550/arXiv.1710.05381.



IVAN V. KONYUKHOV was born in Kazan, Tatar Autonomous Soviet Socialist Republic, USSR, in 1989. He received his candidate degree in physics and mathematics from Kazan Federal University, Russian Federation in 2012. From 2012 till 2017, he had been working as a teaching assistant at the Applied Mathematics Department of Kazan Federal University. Since 2017, he has been a research fellow, postdoctoral researcher, assistant professor and associate professor at Innopolis University, Russian Federation. Nowadays he is also a head of High-Performance Computing Lab. He has authored six teaching aids, over 60 articles and 4 patents. His research interests include computational fluid dynamics, numerical simulations, digital twins, parallel and distributed programming, as well as applied artificial intelligence.

He is also a program committee member of the International Conference on Computational Optimization (ICOMP).



VLADIMIR M. KONYUKHOV was born in Batumi, Georgia Soviet Socialist Republic, USSR, in 1956. He received his candidate and doctoral degrees in physics and mathematics from Kazan State University, USSR in 1988 and 2005, respectively. From 1984, he was a leading research fellow at the Chebotarev Institute of Mechanics and Mathematics and an associate professor at the Department of Applied Mathematics at the Institute of Computational Mathematics and Information

Technologies. Since 2006, he has been a full professor at the department of applied mathematics and artificial intelligence at Kazan Federal University. He has authored five books, over 150 articles and more than 10 patents. His research interests include mathematical modeling of multi-phase flow in porous media and oil wells, simulation of pollutant migration in aquifers, numerical analysis of mass transfer in circulatory systems, numerical methods and computer visualization for physical processes, scientific machine learning and applied artificial intelligence.

He is an associate editor of the proceedings of the International Conference on Parallel Computational Technologies and a member of the dissertation council at Kazan Federal University in the specialty "Mathematical Modeling, Numerical Methods and Software Packages" in the field of Physical and Mathematical Sciences.

...